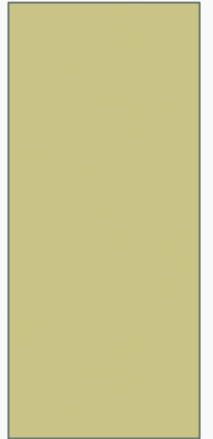


EC-200 DATA STRUCTURES

LECTURE 17



SORTING ALGORITHMS AND THEIR EFFICIENCY

- **Sorting is a Process that Organizes a Collection of Data into Either Ascending or Descending Order**

SELECTION SORT

- To Sort an Array into Ascending Order, **First Search for the Largest Element**
- Because You Want the Largest Element to be in the Last Position of the Array, You **Swap the Last Item With the Largest Item** to be in the Last Position of the Array, You Swap the Last Item with the Largest Item, Even if These Items Appear to be Identical
- **Now, Ignoring the Last (Largest) Item of the Array, search Rest of the Array For Its Largest Item and Swap it With Its Last Item, Which is the Next-to-Last Item in the original Array**
- You Continue Until You Have Selected and Swapped $N-1$ of the N Items in the Array
- The Remaining Item, Which is Now in the First Position of the Array, is in its Proper Order

SELECTION SORT

5	1	3	4	6	2
---	---	---	---	---	---



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Largest



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Largest



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Largest



Comparison

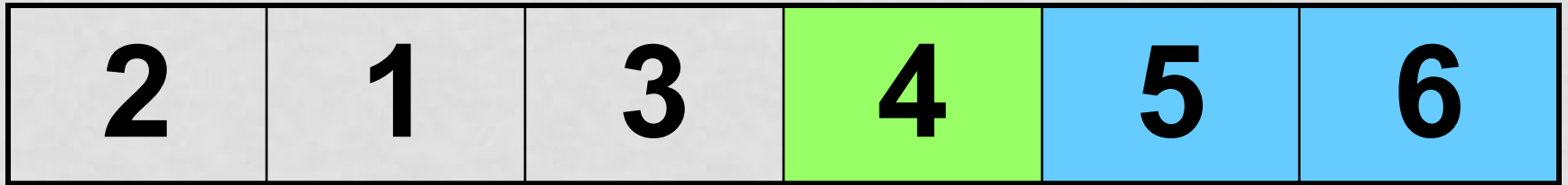


Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



Comparison

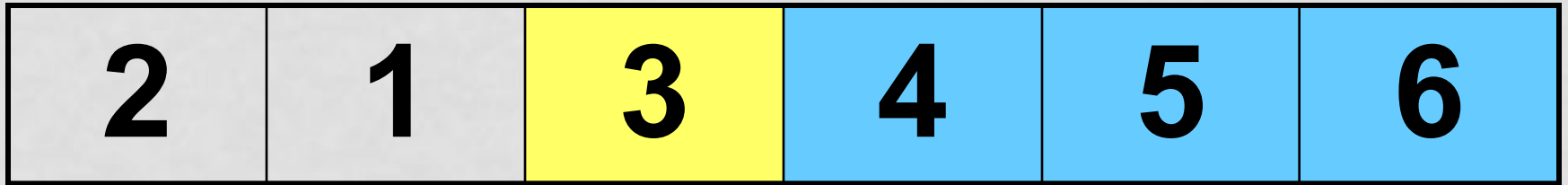


Data
Movement



Sorted

SELECTION SORT



Comparison



Data
Movement



Sorted

SELECTION SORT



□
Largest



Comparison

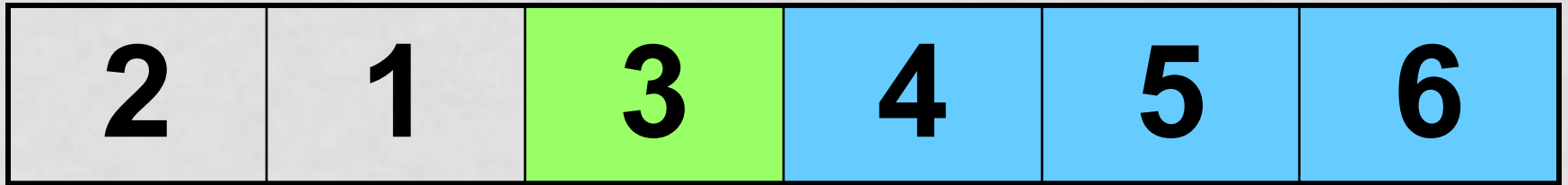


Data
Movement



Sorted

SELECTION SORT



Comparison

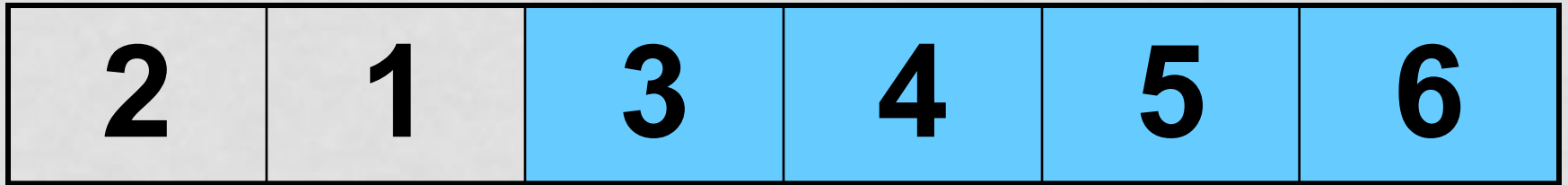


Data
Movement



Sorted

SELECTION SORT



Comparison

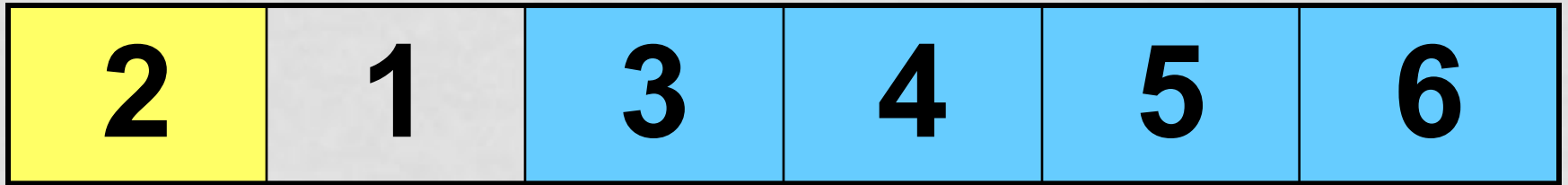


Data
Movement



Sorted

SELECTION SORT



Comparison

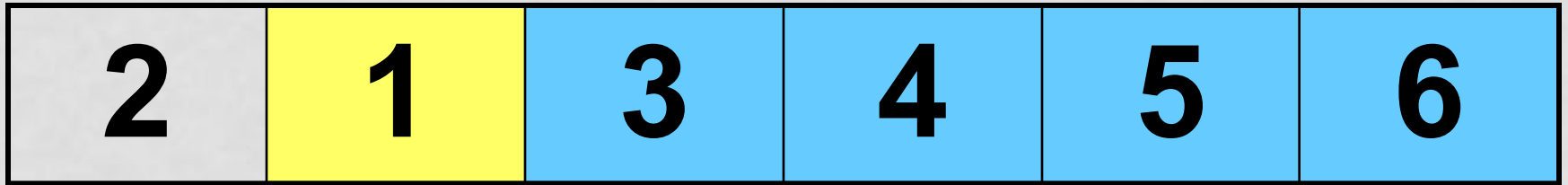


Data
Movement



Sorted

SELECTION SORT



Comparison

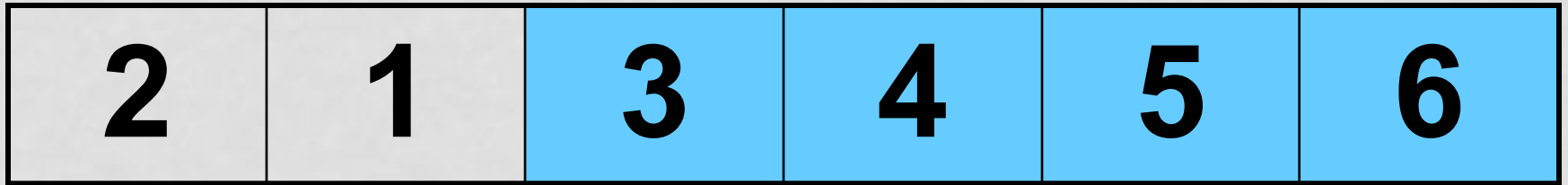


Data
Movement



Sorted

SELECTION SORT



Largest



Comparison

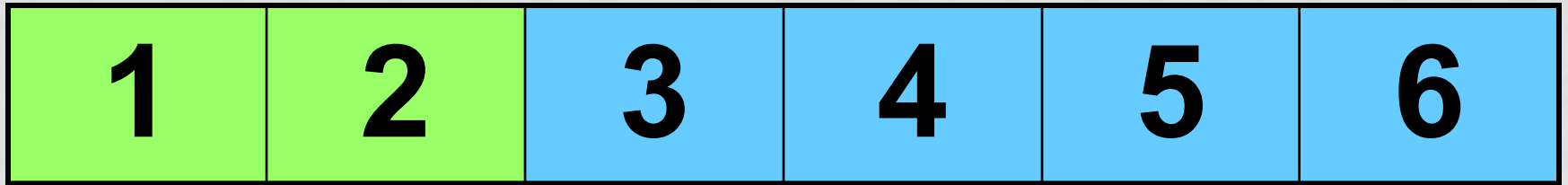


Data
Movement



Sorted

SELECTION SORT



Comparison

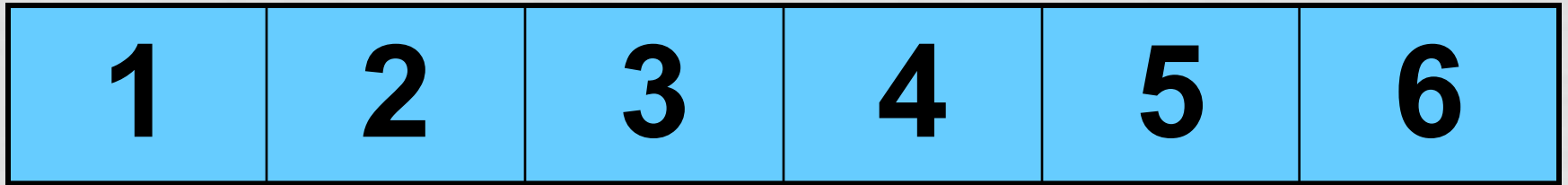


Data
Movement



Sorted

SELECTION SORT



DONE!



Comparison



Data
Movement



Sorted

**Shaded Elements are Selected
Elements in Order are Coloured Red**

Initial Array

29	10	14	37	13
----	----	----	----	----

After1st Swap

29	10	14	13	37
----	----	----	----	----

After2nd Swap

13	10	14	29	37
----	----	----	----	----

After3rd Swap

13	10	14	29	37
----	----	----	----	----

After4th Swap

10	13	14	29	37
----	----	----	----	----

ANIMATION SOURCE

- <http://cse.iitkgp.ac.in/~pds/notes/swf/selection.htm>
!

ANALYSIS

- Selecting the lowest element requires scanning all n elements (this takes $n - 1$ comparisons) and then swapping it into the first position.
- Finding the next lowest element requires scanning the remaining $n - 1$ elements and so on,
- for $(n - 1) + (n - 2) + \dots + 2 + 1$
- $= n(n - 1) / 2 \in \Theta(n^2)$ comparisons
- Arithmetic Progression

BUBBLE SORT

- Compares Adjacent Items and Exchanges Them if They are Out of Order
- When You Order Successive Pairs of Elements, the Largest Element Bubbles to the Top(end) of the Array
- Bubble Sort (Usually) Requires Several Passes Through the Array

BUBBLE SORT

Pass 1

29	10	14	37	13
10	29	14	37	13
10	14	29	37	13
10	14	29	37	13
10	14	29	13	37

Pass 2

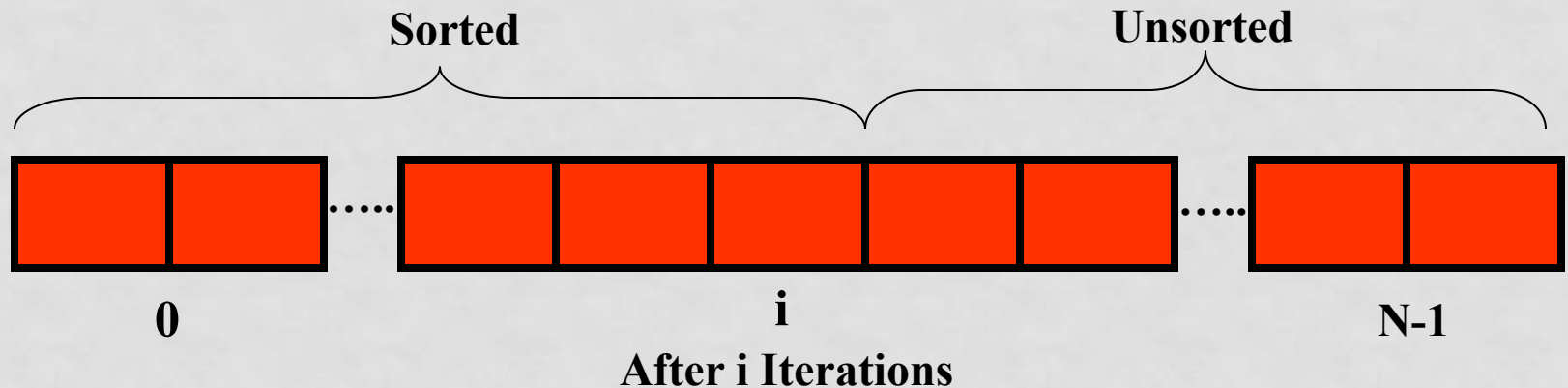
10	14	29	13	37
10	14	29	13	37
10	14	29	13	37
10	14	13	29	37

ANALYSIS

- Not a practical sorting algorithm when n is large.
- The only significant advantage is its ability to detect sorted lists
- The positions of the elements in bubble sort will play a large part in determining its performance.
- Large elements at the beginning of the list do not pose a problem, as they are quickly swapped.
- Small elements towards the end, however, move to the beginning extremely slowly.
- Hence, such elements are referred to as Rabbits and Turtles

INSERTION SORT

- Divide Array Into Sorted Section At Front (Initially Empty), Unsorted Section At End
- Step Iteratively Through Array, Moving Each Element To Proper Place In Sorted Section

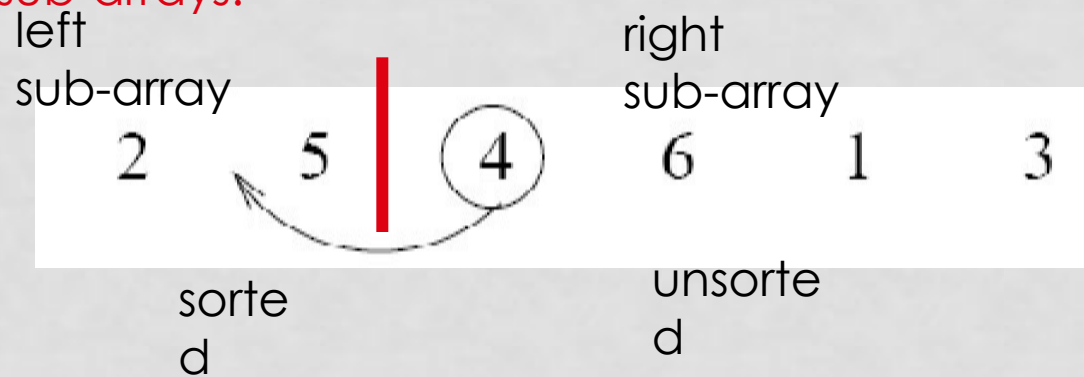


INSERTION SORT

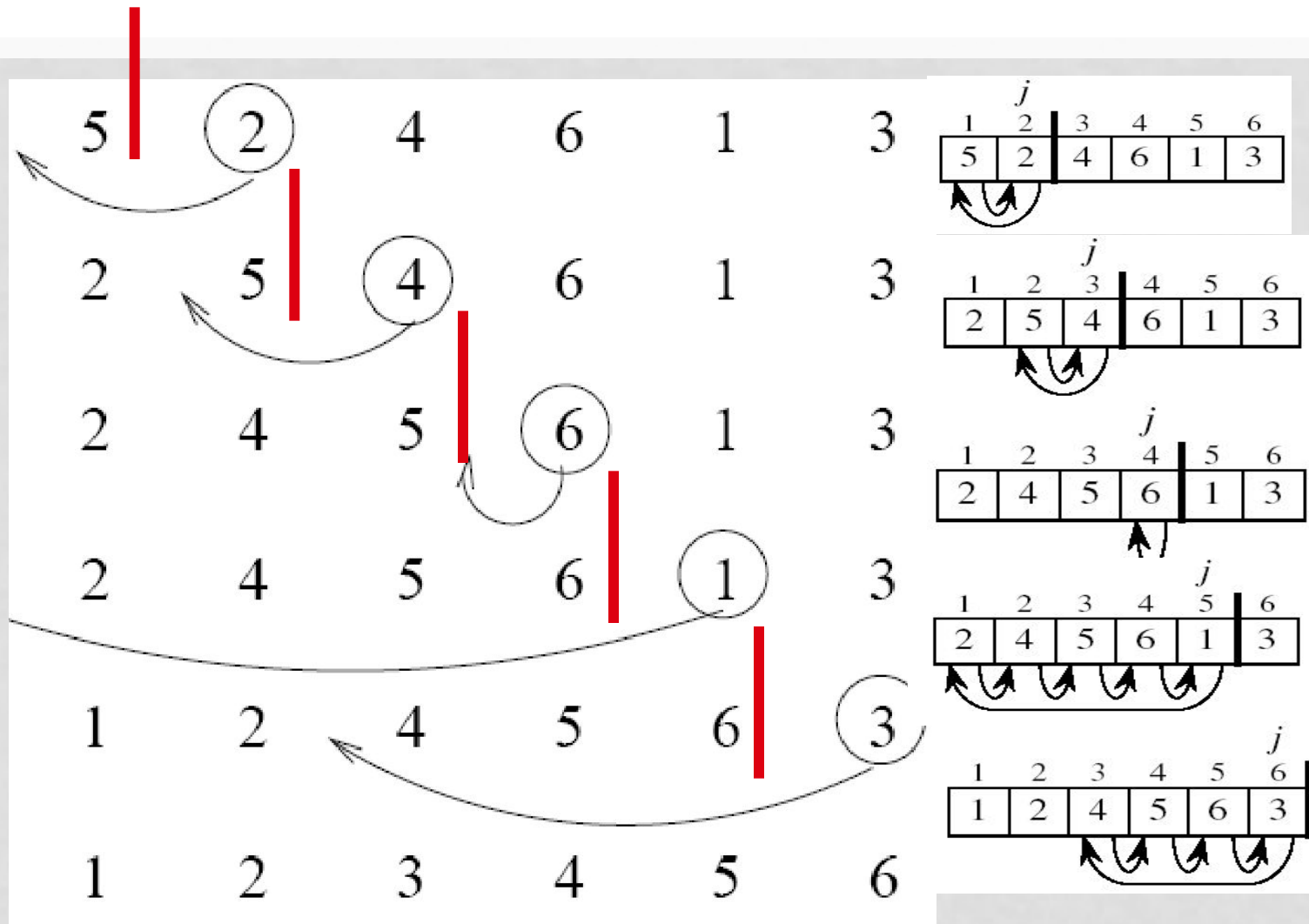
input
array

5 2 4 6 1 3

at each iteration, the array is divided in two
sub-arrays:



INSERTION SORT



WEBSITE SOURCE

- <http://www.ee.ryerson.ca/~courses/coe428/sorting/insertionsort.html>

ANALYSIS

- Best Case
 - Already Sorted
 - $\sim O(n)$
- Worst Case
 - $O(n^2)$

MERGESORT

- Divide and Conquer Algorithm
- Recursively split the input in half
- Then recursively merge pairs of pieces

Recursive Steps:

Divide the Array in Half

Sort the Halves

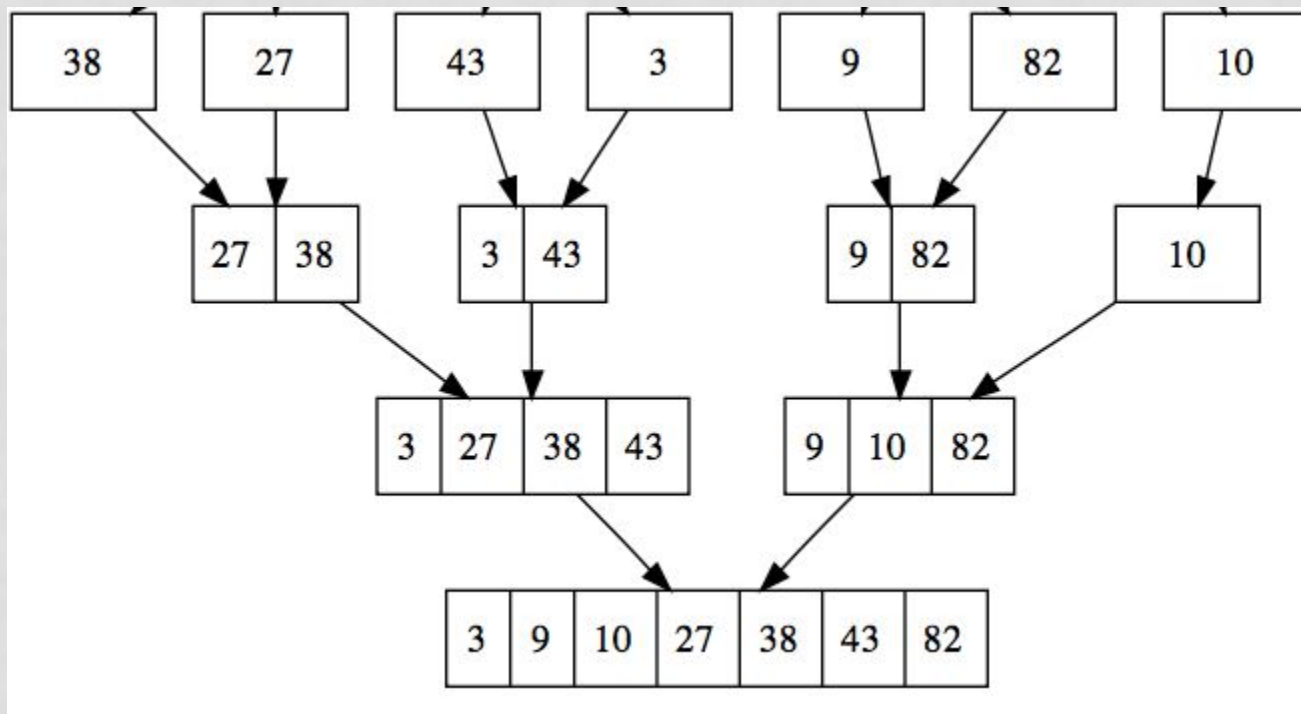
Merge The Halves inside a Temporary Array

Copy Temporary Array to the appropriate locations in original array

MERGESORT

- The Recursive calls Continue to divide the Array into Pieces Until Each Piece Contains Only One Item
- An Array of One Item is Obviously Sorted
- The Algorithm Then Merges These small Pieces Until One Sorted Array Results

MERGE SORT



ANALYSIS

- There are a total of $\log_2 n$ passes
- Each pass requires atmost n comparison
- Complexity = $O(n \log_2 n)$

QUICKSORT

- **Divide and Conquer** algorithm
- Quicksort Works by Partitioning the Array into Two Pieces Separated by a Single Element That is Greater Than all the Elements in the Left part and Smaller Than all the Elements in the right part
- This Guarantees That, the Single Element , Called the **Pivot** Element, is in its Correct position
- Then the Algorithm Proceeds, Applying the Same Method to the Two parts Separately

QUICKSORT

- Partition (Divide)
 - Choose a **pivot**
 - Find the position for the pivot so that
 - all elements to the left are less
 - all elements to the right are greater

< pivot

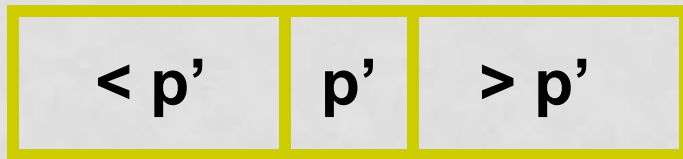
pivot

>= pivot

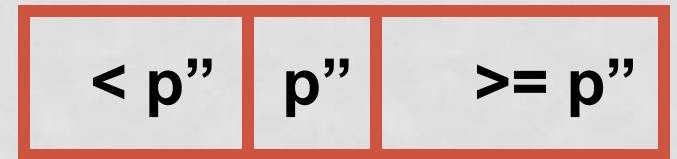
QUICKSORT

- Conquer
 - Apply the same algorithm to each half

< pivot

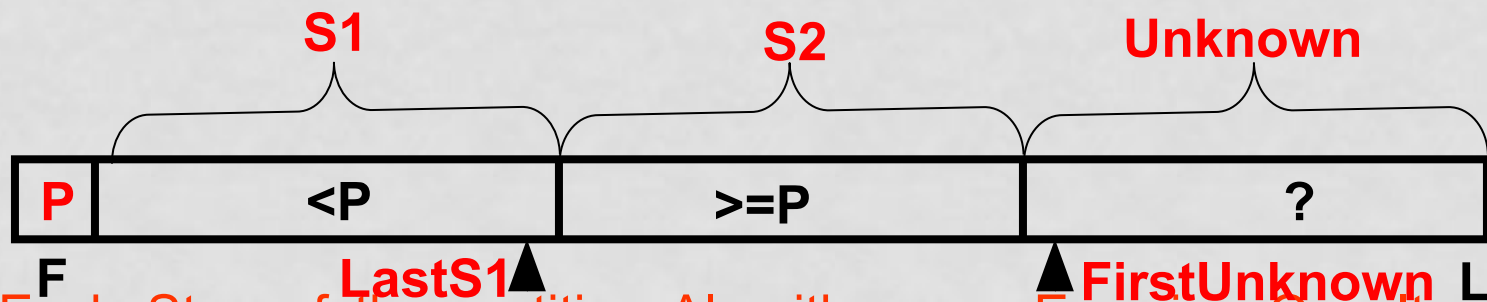


>= pivot



PARTITION ALGORITHM

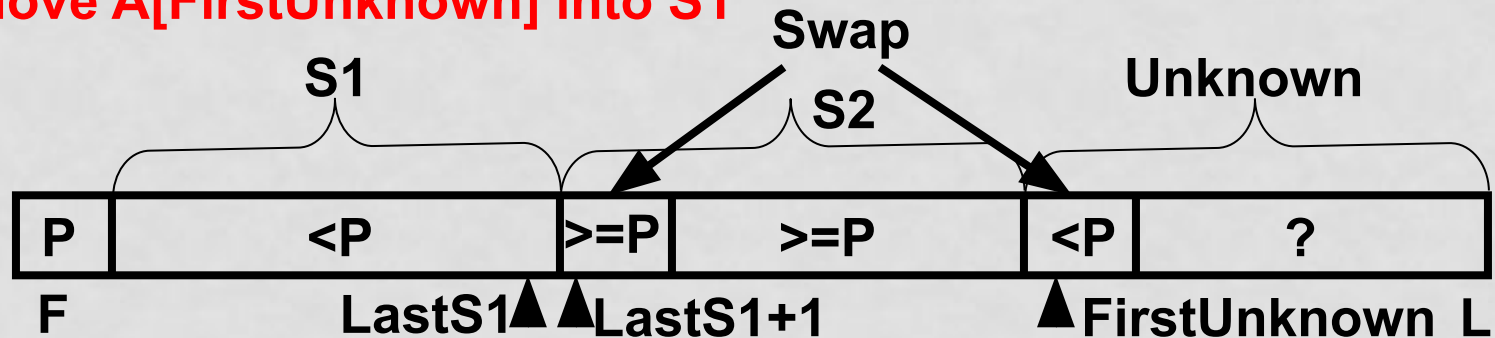
- Must Arrange Items Into Two regions S1, the Set of Items Less Than the Pivot, and S2, the Set of Items Greater Than or Equal to Pivot
- Different algorithms for Choice of a Pivot
- Retain Pivot in A[F] position
- The Items That await Placement are in Another Region , Called the Unknown Region



- At Each Step of the partition Algorithm you Examine One Item from Unknown Region and Place it in S1 or S2

HOW TO PERFORM THE MOVES

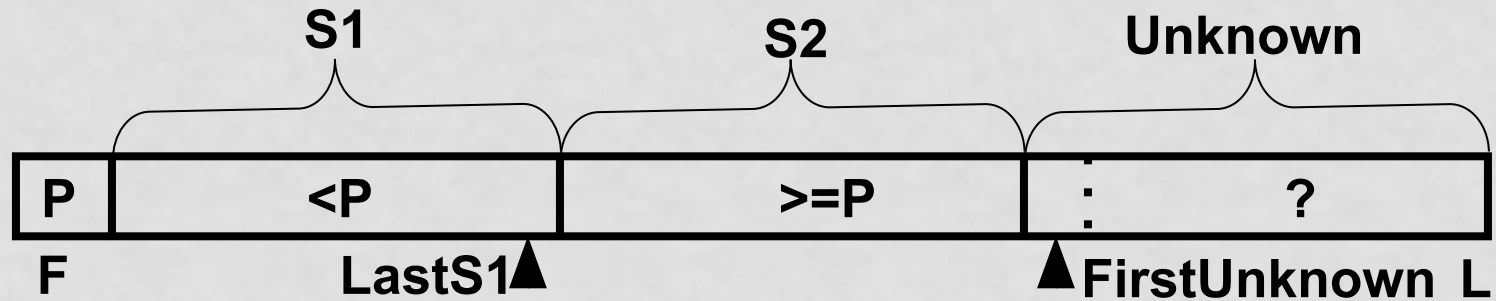
- **To Move $A[\text{FirstUnknown}]$ into $S1$**



- Swap $A[\text{FirstUnknown}]$ With the First Item of $S2$, Which is $A[\text{LastS1}+1]$, and Then Increment $S1$ by 1
- Thus the Item That Was in $A[\text{FirstUnknown}]$ will be at the Rightmost Position of $S1$
- Item of $S2$ That was Moved to $A[\text{FirstUnknown}]$: If you Increment FirstUnknown by 1, That Item Becomes the Rightmost Member of $S2$
- Thus, Statements for the Following Steps are Required
 - **Swap $A[\text{FirstUnknown}]$ with $A[\text{lastS1}+1]$**
 - **Increment LastS1**
 - **Increment FirstUnknown**

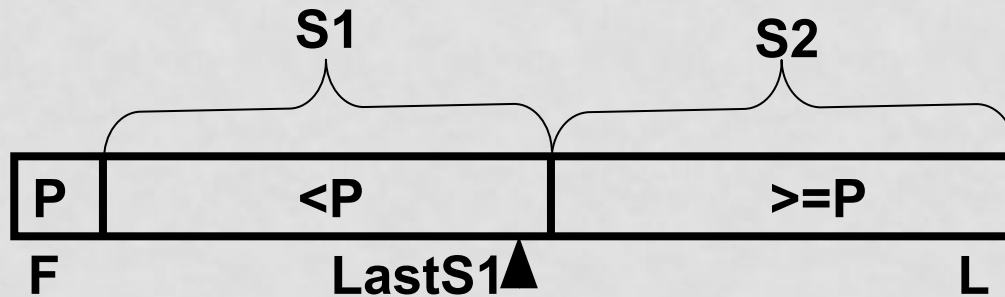
HOW TO PERFORM THE MOVES

- To Move $A[\text{FirstUnknown}]$ into S2



- Region S2 and Unknown are Adjacent
- Simply **Increment FirstUnknown by 1**, S2 Expands to the Right

HOW TO PERFORM THE MOVES



- **To Move Pivot Between S1 and S2**
 - **Interchange $A[\text{LastS1}]$ (i.e. the Rightmost Item in S1) with Pivot**
 - **Thus, Pivot Would be in its Appropriate Location**

Pivot					
27	38	12	39	27	16

Choose Pivot, here A[F],
LastS1=F

Pivot	Unknown				
27	38	12	39	27	16

FirstUnknown = 1(Index to 38
38 Belongs in S2)

Pivot	S2	Unknown			
27	38	12	39	27	16

S1 is Empty
12 Belongs in S1, swap38 & 12

Pivot	S1	S2	Unknown		
27	12	38	39	27	16

39 Belongs in S2

Pivot	S1	S2	Unknown		
27	12	38	39	27	16

27 Belongs in S2

Pivot	S1	S2	Unknown		
27	12	38	39	27	16

16 Belongs in S1, Swap 38 & 16

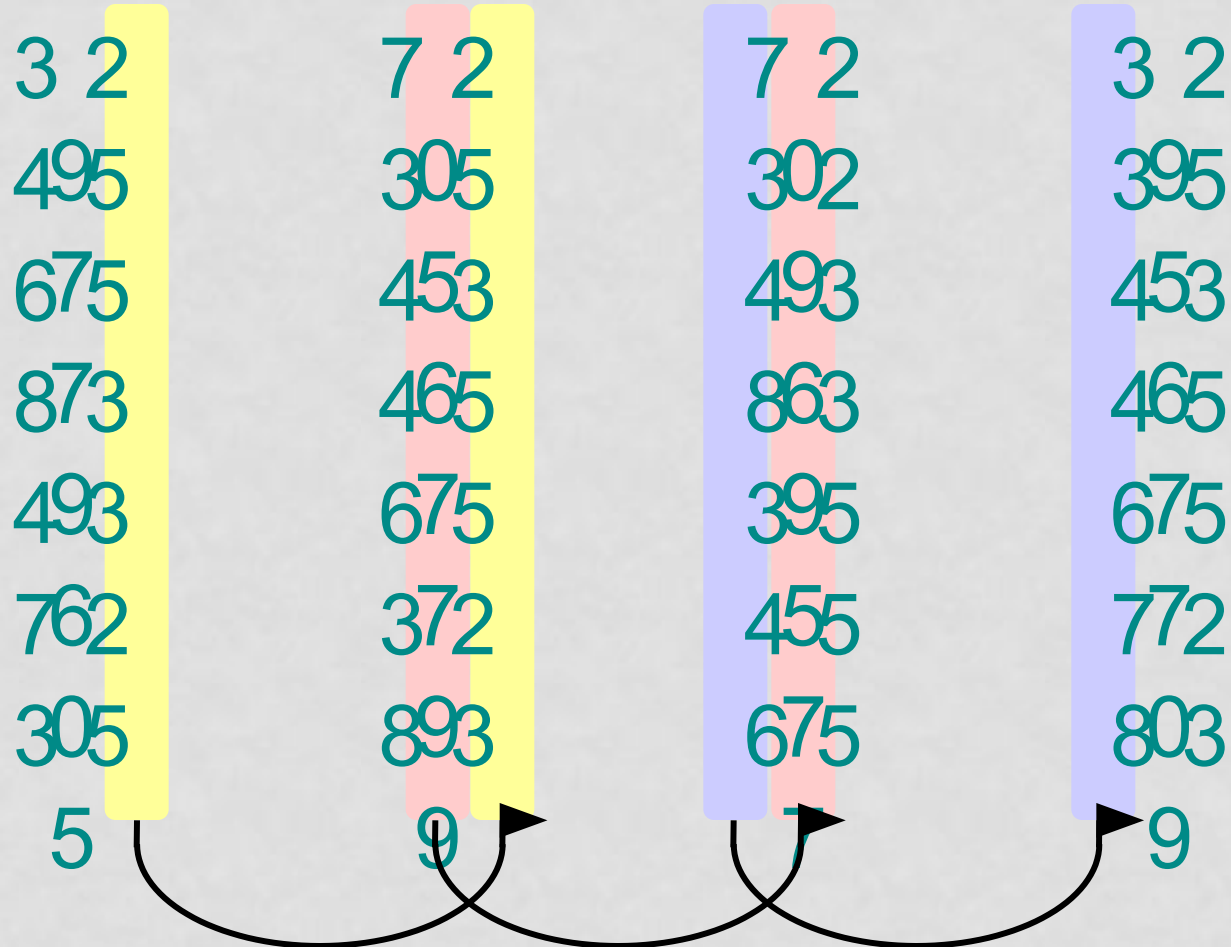
Pivot	S1	S2			
27	12	16	39	27	38

No more Unknown

S1	Pivot	S2			
16	12	27	39	27	38

Place Pivot between S1 and S2

RADIX SORT



COMPARISON OF SORTING ALGORITHMS

<u>Sorting Algo</u>	<u>Worst Case Efficiency (Complexity)</u>
Selection Sort	N^2
Bubble Sort	N^2
Insertion Sort	N^2
Mergesort	$N * \log N$
Quicksort	N^2
Radix Sort	N

BINARY SEARCH

```
int BinarySearch( const int b[ ], int searchKey, int low, int high)
{
    int middle;
    while ( low <= high ) {
        middle = ( low + high ) / 2;
        if ( searchKey == b[ middle ] ) // match
            return middle;
        else if ( searchKey < b[ middle ] )
            high = middle - 1;        // search low end of array
        else
            low = middle + 1;         // search high end of array
    }
    return -1; // searchKey not found }
}
```

EFFICIENCY OF SEARCHING ALGORITHMS

Sequential Search

- In a Sequential Search of an Array of N Items, You Look at Each Item in Turn, Beginning With the First One, Until Either You Find the Desired Item or You Reach the End of the Data Collection
- Best Case : $O(1)$
- Worst Case : $O(N)$

Binary Search

- Binary Search Searches a Sorted Array for a Particular Item
- Suppose $N=2^k$, then in the worst case You Will Perform k Divisions Until Only One Item Remains, as $(N/2^k)=1$
- Hence $k = \log N$
- Worst Case : $O(\log_2 N)$

READING

- Chapter 11, Data Abstraction and Problem Solving using C++, Walls & Mirrors 6th ed.